

1. Estructura general del proyecto

El proyecto se compone de dos partes principales:

Parte 1 (kurtz.py + kurtz_colors.py): entorno de cuadrícula con peligros (precipicios y soldado), un objetivo (Kurtz) y una salida. El agente (capitán) razona a partir de perceptos locales, mantiene una **Base de Conocimiento (KB)** y planifica con **BFS, DFS, GBFS y A***, generando además una traza evaluable del proceso de búsqueda.

Parte 2 (palacio.py + river_mdp.py):

- **Palacio Bayesiano (palacio.py):** el agente mantiene **creencias probabilísticas** sobre la ubicación de trampas, militar, salida y Kurtz. Actualiza con **Bayes** (likelihood 0/1), construye mapas numéricos de riesgo y, en modo automático, planifica rutas bajo un **umbral de riesgo p**.
- **Cruce del río como MDP (river_mdp.py):** modelado como **Proceso de Decisión de Markov**, resolución con **Value Iteration** (ecuación de Bellman óptima), extracción de política y simulación de episodios.
- Cabe destacar que se ha implementado un sistema de semillas para que se pueda ejecutar varias veces el mismo mapa.

2. PARTE 1 — “Buscando al Coronel Kurtz” (kurtz.py)

2.1. Mundo real y perceptos

El mundo es una cuadrícula $n \times n$ con:

- **exactamente 3 precipicios,**
- **1 soldado,**

- **1 salida,**
- **Kurtz,**
todos en celdas distintas y distintas de la celda inicial (1,1).

El percepto del agente es un vector de 8 booleanos, en este orden exacto:

1. **Brisa:** True si hay algún precipicio en una celda adyacente (4-conexa).
2. **Ronquido:** True si el soldado está vivo y está en una celda adyacente.
3. **Resplandor:** True si la salida está en la celda actual o en una celda adyacente.
4. **Pared $\uparrow$:** True si el agente está en el borde superior.
5. **Pared $\downarrow$:** True si está en el borde inferior.
6. **Pared $\leftarrow$:** True si está en el borde izquierdo.
7. **Pared $\rightarrow$:** True si está en el borde derecho.
8. **Grito:** True únicamente en el turno posterior a matar al soldado con la granada (y luego se “consume”).

Este diseño permite separar claramente:

información local de peligros (brisa/ronquido),

Información local de objetivo (resplandor),

Límites del mundo (paredes),

Y el resultado de una acción activa de comprobación (grito tras granada).

2.2. Base de Conocimiento (KB): qué guarda y por qué

La KB es un diccionario que almacena solo lo que el agente aprende por perceptos y acciones, sin “mirar el mapa real”. Entre los campos clave:

visited: celdas visitadas físicamente.

breeze_obs, snore_obs, glow_obs: observaciones de perceptos en celdas visitadas.

pit_worlds: lista de mundos posibles (modelos) de precipicios compatibles con lo observado, imponiendo “exactamente 3 precipicios”.

soldier_cand, exit_cand: conjuntos de candidatos compatibles con ronquidos/resplandores.

soldier_forbidden: conocimiento negativo explícito de celdas donde el soldado no puede estar (muy importante tras fallar una granada).

soldier_known, exit_known: si queda un único candidato, se fija como conocido para mejorar consistencia y claridad.

Extra implementado (importante): inferencia con “punto fijo”

Las inferencias (precipicios, soldado, salida) tienen dependencias cruzadas. Por ejemplo:

- si se deduce la salida con certeza, esa celda no puede ser precipicio,
- y si se deduce un precipicio seguro, esa celda no puede ser salida ni soldado.

Para propagar estas restricciones dentro del mismo “turno lógico”, se implementa una iteración corta hasta estabilidad (máximo 3 iteraciones), recalculando: `Infer_pits(kb)`, `Infer_soldier(kb)` e `infer_exit(kb)`, hasta que no cambie el estado relevante. Esto es más sólido que hacer inferencia “una sola pasada”.

2.3. Inferencia de precipicios: enumeración consistente con exactamente 3 precipicios

Esta es una de las partes más relevantes (y donde más se nota el trabajo extra). Hechos lógicos de partida (por observaciones de brisa) Para cada celda visitada v :

1. Si $Brisa(v)=True$, entonces existe al menos un vecino con precipicio
2. Si $Brisa(v)=False$, entonces ningún vecino puede ser precipicio:

Cálculo implementado: “mundos posibles” de precipicios

En lugar de marcar candidatos de forma local únicamente, el código:

1. Construye el conjunto de celdas candidatas a precipicio: no visitadas, no descartadas por $brisa=False$, y además excluye celdas conocidas como salida o soldado (si ya se dedujeron).
2. Enumera todas las combinaciones de **3** celdas candidatas.
3. Filtra las combinaciones que violen alguna restricción:

para cada brisa=True, la combinación debe intersectar su vecindario,

para cada brisa=False, la combinación no debe intersectar su vecindario.

4. El conjunto resultante se guarda en pit_worlds.

A partir de pit_worlds se definen: Precipicio posible (Pit?): celdas que aparecen en algún mundo válido y Precipicio seguro (Pit!): celdas que aparecen en todos los mundos válidos.

Se ha hecho uso de este enfoque ya que, respeta la restricción global “exactamente 3”, algo que una inferencia puramente local no haría bien.

2.4. Inferencia del soldado y de la salida (candidatos consistentes)

INFERENCIA SOLDADO (si está vivo)

Se parte de todas las celdas, y se aplican restricciones:

1. No puede estar en visitadas (si lo pisas, mueres; el agente sigue vivo).
2. No puede estar en precipicios seguros (PitMust).
3. No puede estar en la salida si esta está deducida.
4. Por cada observación de ronquido en v:

si ronquido=True, el soldado debe estar en Adj(v),

si ronquido=False, el soldado no puede estar en Adj(v).

Esto se implementa como: “zonas prohibidas” = unión de celdas contiguas de ronquido=False, y “zonas obligatorias” = intersección de celdas contiguas de ronquido=True (si hay varias).

5. Extra clave: soldier_forbidden descarta celdas donde se lanzó granada y no hubo grito (si el soldado sigue vivo).
6. Si tras todo esto queda un único candidato, se fija soldier_known.

SALIDA

La salida se deduce de glow_obs con una lógica similar, si se observa esplendor, significa que la salida se encuentra en una casilla contigua y sino, significa que la salida no puede estar en una contigua. Además, se excluye precipicios seguros y soldado si este ya está deducido con certeza.

2.5. Etiquetado y visualización del mapa (extra muy útil)

Se implementa una etiqueta por celda que permite distinguir:

- **v** si está visitada,
- **✓** si es segura deducida aunque no visitada,
- **.** si es desconocida,
- **P? / P!** para precipicio posible/seguro,
- **S? / S!** para soldado posible/seguro (solo si está vivo),
- **E? / E!** para salida posible/deducida.

Extra implementado: overlays y persistencia de salida

La celda del agente se imprime como CW con subíndices de estímulos (B/R/E).

Si Kurtz ha sido encontrado, pasa a CWK.

Si el agente está sobre la salida, se marca explícitamente con CWE / CWKE.

Si alguna vez se pisó la salida real, esa celda queda marcada persistentemente como E aunque el agente se mueva.

Todo esto, junto con **colores** (ver kurtz_colors.py), mejora enormemente la interpretabilidad de la ejecución.

2.6. Búsqueda: BFS, DFS, GBFS y A* con traza evaluable (Frontier / Removed / Explored)

La planificación se implementa en `plan_path_virtual_kb(...)`, y tiene una característica crucial:

Extra implementado: “visita virtual” durante planificación

El planificador **no mueve al agente real**, pero cuando extrae un nodo **u** de la frontera, consulta el percepto real en `u` (`percept_at(world, u)`), actualiza la KB como si se hubiera observado, y solo expande vecinos demostrablemente seguros según `is_safe(kb, v)`.

Esto permite que el planificador vaya “aprendiendo” durante la búsqueda (a nivel de planificación), sin ejecutar físicamente movimientos.

Traza

En cada extracción se imprime la frontera, el nodo removido y los extraídos o visitados, al igual que hacíamos en clase

Con formato consistente. En GBFS y A* se imprime además el valor asociado (h o f).

Detalles implementados para determinismo (extra clave)

Los vecinos se expanden en un **orden fijo de direcciones (FRONTIER_DIR_ORDER)**.

En GBFS/A* se añade un contador incremental en la frontera (tuplas del tipo (valor, counter, celda)) para desempatar de manera estable. Esto es importante: hace que la traza sea reproducible y no dependa del comportamiento interno de estructuras no ordenadas. Es decir, para una misma semilla da siempre la misma salida.

3. Por qué la heurística Manhattan es admisible (y también consistente)

En GBFS y A* se usa la distancia Manhattan:

$$h(n) = |r_n - r_{\text{goal}}| + |c_n - c_{\text{goal}}|$$

3.1. Admisibilidad (no sobreestima)

En una mapa con coste 1 por movimiento, cualquier camino desde n hasta el objetivo debe:

- corregir al menos recorrer al menos $|r_n - r_{\text{goal}}|$ diferencias verticales,
- y al menos $|c_n - c_{\text{goal}}|$ diferencias horizontales.

Cada acción cambia como máximo 1 unidad en una de esas direcciones, así que el número real mínimo de pasos $d(n)$ cumple que $d(n) \geq h(n) = |r_n - r_{\text{goal}}| + |c_n - c_{\text{goal}}|$

3.2. Consistencia (monótona)

Para dos estados vecinos n y n' (separados por un movimiento), la Manhattan al objetivo cambia como mucho en 1: $h(n) \leq 1 + h(n')$.

como el coste del paso es $c(n, n') = 1$, se cumple: $h(n) \leq c(n, n') + h(n')$

4. Extra principal de la Parte 1: Modo automático avanzado (kurtz.py)

Además del modo manual, se implementa `run_auto(...)`, que automatiza la misión en dos fases: Planifica y ejecuta un camino hasta Kurtz y Planifica y ejecuta un camino hasta la salida.

Extra implementado: KB de planificación separada

Durante la planificación se usa una KB separada (`kb_plan`) para no “contaminar” la KB real de ejecución hasta que el plan exista. La ejecución real, en cambio, actualiza la KB del agente conforme se mueve.

Extra implementado: gestión explícita de bloqueos

Si durante la planificación no quedan celdas demostrablemente seguras para expandir, el sistema: detecta candidatos “de riesgo” en la frontera de lo explorado, **excluye** celdas con muerte segura (!) y solo propone celdas con incertidumbre (?), avisa explícitamente al usuario del riesgo y permite (si el usuario lo decide) forzar la expansión de una celda no segura para desbloquear nueva información.

Esto está por encima de lo mínimo exigible: no se limita a “no hay solución”, sino que implementa una estrategia controlada para continuar bajo riesgo.

5. PARTE 2A — Palacio Bayesiano (palacio.py)

En esta parte el agente se enfrenta a un entorno probabilístico: no conoce con certeza la ubicación de los elementos del mundo (trampas, militar, salida y Kurtz), sino que **mantiene y actualiza creencias** sobre su posible localización a medida que recibe perceptos. Cada movimiento aporta nueva información local y el agente utiliza el **Teorema de Bayes** para actualizar su conocimiento, generando mapas de probabilidad y de riesgo que guían su planificación.

5.1. Representación de creencias

Para cada entidad $\tau \in \{F, P, D, M, S, CK\}$ se almacena una matriz de creencias $Bel_{\tau}(i, j)$, que indica la probabilidad de que dicha entidad se encuentre en la celda (i, j) . Cada distribución cumple las condiciones de positividad y normalización:

$$Bel_{\tau}(i, j) \geq 0, \quad \sum_{i,j} Bel_{\tau}(i, j) = 1$$

Estas creencias se actualizan constantemente a medida que el agente explora el palacio.

5.2. Zona de evidencia y perceptos

Cada percepto informa sobre la presencia o ausencia de una entidad en la **zona local** compuesta por la celda actual y sus adyacentes. Matemáticamente, se define:

$$Zone(pos) = Adj(pos) \cup \{pos\}$$

Si el percepto de una entidad τ en la posición actual es positivo ($e^\tau=1$), se sabe que τ se encuentra en esa zona. Si es negativo ($e^\tau=0$), se sabe que τ no se encuentra allí.

5.3. Actualización Bayesiana con verosimilitud determinista

El proceso de actualización aplica una **máscara binaria** sobre la distribución anterior, eliminando las posiciones incompatibles con la evidencia y renormalizando la probabilidad restante. Formalmente:

$$Bel'_\tau(c) \propto Bel_\tau(c) \cdot Mask_\tau(c)$$

donde $Mask_\tau(c)$ vale 1 si la celda c es compatible con el percepto y 0 en caso contrario.

Tras aplicar la máscara, se renormaliza para que la distribución vuelva a sumar 1:

$$Bel'_\tau(c) = \frac{Bel_\tau(c) \cdot Mask_\tau(c)}{\sum_u Bel_\tau(u) \cdot Mask_\tau(u)}$$

Además, todas las celdas visitadas se marcan automáticamente como imposibles para trampas y militar, pues si el agente ha sobrevivido al pisarlas, no pueden contener peligro.

5.4. Restricción estructural: entidades no pueden estar en trampas

El modelo establece que el **militar (M)**, la **salida (S)** y **Kurtz (K)** no pueden ocupar una **celda que contenga una trampa** (de tipo F, P o D). Dado que las trampas pueden coincidir entre sí, el número de celdas seguras varía en cada escenario.

Para tener en cuenta esta relación se define un **prior de seguridad** que representa la probabilidad de que una celda esté libre de trampas.

Primero se calcula el número esperado de celdas seguras cuando las trampas están en posiciones a, b, d

$$safe_count = (N^2 - 1) - |\{a, b, d\}|$$

La probabilidad de que una celda c sea segura, condicionada a esa configuración, es:

$$P(X = c \mid a, b, d) = \frac{\mathbf{1}\{c \notin \{a, b, d\}\}}{safe_count}$$

Para construir el prior completo, se promedia sobre todas las combinaciones posibles de posiciones de las trampas, ponderando por las creencias actuales de Bel_F , Bel_P y Bel_D :

$$q_{safe}(c) = \sum_{a,b,d} Bel_F(a) Bel_P(b) Bel_D(d) \cdot \frac{\mathbf{1}\{c \notin \{a, b, d\}\}}{(N^2 - 1) - |\{a, b, d\}|}$$

Las creencias de M, S y K se obtienen combinando su evidencia (likelihood) con este prior de seguridad:

$$Bel_X(c) \propto X_like(c) \cdot q_{safe}(c)$$

donde $X \in \{M, S, CK\}$.

5.5. Mapas de probabilidad y riesgo

Con las distribuciones actualizadas, el agente calcula distintos mapas derivados que resumen el nivel de peligro o seguridad de cada celda.

(A) Probabilidad de que haya al menos una trampa

Asumiendo independencia entre los tres tipos de trampa, la probabilidad de que haya **alguna trampa** en una celda es:

$$P(Trap)(c) = 1 - (1 - Bel_F(c))(1 - Bel_P(c))(1 - Bel_D(c))$$

(B) Riesgo de muerte al entrar en una celda

Dado que militar y trampa son mutuamente excluyentes en una misma celda, el riesgo total de morir se obtiene sumando ambas probabilidades y truncando a 1:

$$P(muerte)(c) = \min(1, P(Trap)(c) + Bel_M(c))$$

(C) Celdas seguras según un umbral de riesgo

Se considera **celda segura** aquella cuyo riesgo es menor que un umbral p :

$$P(muerte)(c) < p$$

El conjunto de celdas seguras se usa para marcar posiciones con el símbolo ✓ y definir las zonas transitables en la planificación automática.

5.6. Granada y actualización del militar

Cuando el agente lanza una granada a una celda contigua, se produce un **grito** si el militar estaba allí. Si **no hay grito**, esa celda se descarta como posible ubicación del militar:

$$M_like(target) = 0$$

Tras incorporar esta evidencia negativa, se recalcula el posterior de M combinando la nueva evidencia con el prior de seguridad:

$$Bel_M(c) \propto M_like(c) \cdot q_{safe}(c)$$

5.7. Modo automático y control de riesgo

El **modo automático (AUTO)** permite que el agente planifique y ejecute rutas hacia los objetivos basándose en sus creencias probabilísticas. El proceso combina **búsqueda informada** con **actualización continua** de probabilidades.

1. **Planificación activa:** mientras planifica, cada vez que se extrae un nodo de la frontera se observa su percepto real, se actualizan las creencias y se recalcula el conjunto de celdas seguras.
2. **Escalado temporal del umbral:** si no existe ruta con el umbral actual p , se permite incrementarlo temporalmente en pasos (por ejemplo de 0.05) hasta encontrar un camino factible.
3. **Control dinámico de riesgo:** antes de ejecutar un paso, el agente comprueba el riesgo de la celda destino. Si es demasiado alto, puede lanzar una granada o aumentar temporalmente el umbral para continuar.

De esta forma, el agente integra razonamiento probabilístico, planificación adaptativa y acciones reactivas dentro de un mismo marco coherente.

6. PARTE 2B — Cruce del río como MDP (river_mdp.py)

6.1. Modelado formal

Estados: $s=(r,c)$., **Acciones:** up, down, left, right, stay. **Terminales:** salida (y opcionalmente islas si `deadly_islands=True`).

Formalmente, el espacio de estados es:

$$S = \{(r, c) \mid 0 \leq r < rows, 0 \leq c < cols\}$$

y el conjunto de acciones es:

$$A = \{\text{up, down, left, right, stay}\}$$

6.2. Transiciones con corriente (cálculo exacto)

Para una columna c , se define la fuerza de corriente:

$$p_{\text{down}} = \text{river_strength}(c)$$

y la probabilidad de seguir la dirección elegida (cuando no es “down”) es:

$$p_{\text{dir}} = 1 - p_{\text{down}}$$

La regla esencial implementada es:

Si $a=\text{down}$, el intento es bajar con probabilidad 1 (si es válido; si no, se queda).

Si $a \neq \text{down}$, entonces: con probabilidad p_{dir} se intenta el movimiento elegido, con probabilidad p_{down} se baja (empuje de la corriente), y si alguno de esos destinos es inválido (borde o isla obstáculo), esa masa de probabilidad se convierte en “stay”.

6.3. Recompensas

La recompensa por transición se compone de:

1. coste por paso:

$$r += \text{step_cost}$$

2. bonus por llegar a la salida:

$$r += \text{reward_exit} \quad \text{si } ns = \text{exit}$$

3. penalización por entrar en isla si son mortales:

$$r += \text{reward_island} \quad \text{si } \text{deadly_islands} = \text{True} \text{ y } ns \in \text{islands}$$

Por defecto: $\text{step_cost}=-1$, $\text{reward_exit}=+100$, $\text{reward_island}=-100$.

6.4. Value Iteration: ecuación de Bellman y criterio de parada

Se calcula el valor $V(s)$ iterando la ecuación óptima.

Para cada estado y acción:

$$Q(s, a) = \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma V(s') \right)$$

Actualización:

$$V_{\text{new}}(s) = \max_{a \in A} Q(s, a)$$

Medida de convergencia (cambio máximo):

$$\delta = \max_s |V_{\text{new}}(s) - V(s)|$$

Parada cuando:

$$\delta < \theta$$

El programa incorpora además una nota práctica: con $\gamma=1$ la convergencia puede ser lenta o no alcanzar θ en el número máximo de iteraciones, y en ese caso se devuelve la mejor aproximación disponible.

6.5. Extracción de política óptima

Una vez calculado V , se define la política greedy:

$$\pi(s) = \arg \max_{a \in A} Q(s, a)$$

6.6. Simulación de episodios y retorno acumulado

En la simulación, el agente sigue la política π y se muestrea el siguiente estado según $P(\cdot | s, a)$. El retorno acumulado se calcula como:

$$G = \sum_{t=0}^T \gamma^t r_t$$

Si $\gamma=1$, esto se reduce a suma simple de recompensas.

Al final se reporta: número de éxitos (llegadas a salida), pasos medios y retorno medio (puntos).

6.7. Extras en river_mdp.py

1. **Parámetros configurables de instancia:** tamaño, semilla, número de islas, rango de corriente, descuento γ , recompensas.
2. **Chequeo BFS para evitar mapas imposibles:** al colocar islas aleatoriamente se comprueba que exista al menos un camino geométrico start→exit evitando islas (sin considerar corriente). Esto evita instancias triviales sin solución por mala generación.

3. **Acción stay incluida explícitamente** para completar el modelo y manejar situaciones donde moverse no es viable.
4. **Variante opcional de islas mortales:** permite entrar en islas y terminar con penalización (en lugar de tratarlas siempre como obstáculo).
5. **Salida detallada y didáctica:** impresión del mapa, valores, política y trazas de episodios (con distribución de transición visible) para justificar resultados.